



Data Pipeline Engineering

*AICB-P2T2 · Ngày 17 · Chương 4: Hạ
Tầng · Xây đường ống dữ liệu nuôi AI*

Giảng viên

VinUniversity · Phase 2 · Track 2 · Tuần 4

HÃY SUY NGHĨ...

*“Team X train model trên data có 30% duplicate records. Model memorize duplicates → hallucinate in production. 2 tuần + \$8K GPU lãng phí — chỉ vì pipeline thiếu một bước dedup. **Pipeline của bạn đang nuôi model, hay đang đầu độc nó?**”*

Giữ câu hỏi này trong đầu khi học bài hôm nay

1. Vì sao pipeline quyết định số phần model
2. ETL vs ELT & Medallion Architecture
3. Ingestion: batch, CDC, dlt, unstructured→embedding
4. Orchestration: Airflow 3 & asset-based (Dagster)
5. Declarative & asset pipelines
6. Kafka & Streaming Ingestion
7. Batch/Streaming & dbt Transform
8. Validation, Quality Gates & Data Contracts
9. AI-Specific (dedup, feature parity, flywheel)
10. Thực hành: Agent trace, RAG ingest, feature store
11. Complex RAG & Knowledge Graph (VLM, ColPali, GraphRAG)
12. Reliability, Cost & Demo

Sau buổi học này, bạn sẽ:

1. Thiết kế ETL/ELT pipeline robust cho AI training & inference data
2. Orchestrate data flow bằng Airflow DAG (và hiểu asset-based alternatives)
3. Implement streaming ingestion với Kafka cho real-time features
4. Áp dụng validation gates & data contracts ngăn data bẩn vào model
5. Biến agent trace & dữ liệu phi cấu trúc thành dataset (dedup, decontaminate) nuôi eval/fine-tune & RAG/KG

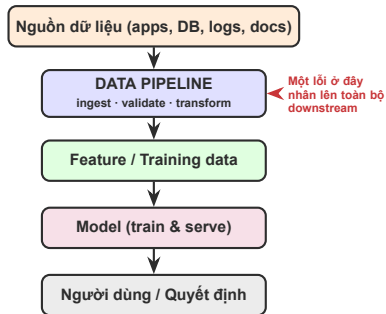
Pipeline mindset + ETL/ELT/Medallion (40') → Ingestion & Orchestration (50') → Streaming & Transform (50') → Quality, Contracts & AI pipelines (40') → Thực hành Agent/RAG & Complex RAG/KG (40') → Demo & Lab

Pipeline chạy được: ingest → validate → transform → load, với dedup + quality gate + dbt tests pass

- Orchestrated DAG 4+ tasks: extract → validate → transform → load
- Medallion layers (Bronze/Silver/Gold) trên DuckDB, dedup ở Silver
- Pandera/GX validation suite + quarantine cho bad records
- dbt-duckdb project với staging & gold models, dbt test pass
- (Bonus) streaming event sim + unstructured→embedding ingestion

- Ai cũng import transformers — kiến trúc model gần như miễn phí
- Khác biệt thật nằm ở **data** bạn đưa vào
- “Garbage in → garbage out” áp dụng gấp đôi cho AI

Lưu ý: Pipeline là một dependency của model — chính xác như một thư viện. Pipeline hỏng âm thầm = model hỏng âm thầm, nhưng *accuracy dashboard vẫn xanh*.



Unity (2022)

\$110M

≈8% doanh thu năm

Ad-targeting model ăn
bad data từ 1 khách hàng lớn

→ accuracy sụp đổ

Zillow Offers (2021)

\$304M

write-down Q3 + đóng cửa BU

Model định giá nhà mua hớ
≈7.000 căn → cắt 25% nhân sự
(distribution shift / drift)

Equifax (2022)

2.5M scores

credit scores sai 3 tuần

Code lỗi chưa test trong
scoring pipeline; <300k lệch
≥25 điểm. NY AG phạt \$725k

Mẫu số chung: không ai bị hack. Chỉ là *data xấu đi qua một pipeline thiếu gate*. Nguồn: Coralogix/Monte Carlo (Unity), SEC 8-K (Zillow), NY AG (Equifax).

1 / 10

bảng dữ liệu gặp ≥ 1 sự cố mỗi năm (Monte Carlo 2026, +11M bảng; xấu đi từ 1/15)

\$3M / tháng

chi phí trung bình do pipeline failures (Fivetran 2026, $n=500$); tới \$1.4M/sự cố; ~13h khắc phục

- Pipeline execution faults — **26.2%**
- Real-world variation — 20%
- Intentional changes (backfills) — 14.2%
- Schema drift — 7.8%

Lưu ý: 53% năng lực kỹ sư dữ liệu dành cho *bảo trì pipeline* (Fivetran 2026). Pipeline tốt không phải là tính năng phụ — nó là phần lớn công việc.

Hôm nay ta học cách xây pipeline để **không** trở thành các con số trên. Observability sâu hơn → Ngày 27.

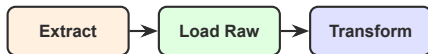
ETL (truyền thống)



EtLT (thực tế: hybrid)



ELT (cloud-native, AI) — mặc định



ELT thắng mặc định: compute lakehouse (Snowflake/-
Databricks/BigQuery) rẻ → transform tại chỗ.
ETL vẫn sống cho: mask PII trước khi load (regulated → Ngày
24), target on-prem yếu compute, transform bằng Python lib.

“ETL is dead” là cường điệu — thực tế phần lớn pipeline là **EtLT**: extract-time transform nhẹ, rồi heavy transform trong warehouse.



Medallion trả lời "tổ chức chất lượng *tăng dần* thế nào" — KHÔNG phải ETL/ELT.
ETL/ELT trả lời "transform chạy ở *đâu*". Thực tế medallion = ELT bên trong.

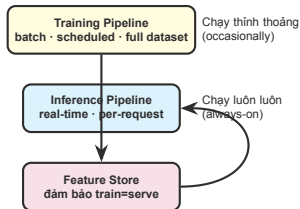
Rule #1: Luôn giữ Bronze (raw) — không bao giờ xoá data gốc. Retraining & reproducibility cần full history. (Table format Delta/Iceberg cho Bronze → Ngày 18.)

Lưu ý: 4 anti-patterns (Data Engineering Weekly, 9/2025)

- **Bronze-misuse:** report thẳng từ raw
- **Silver-overload:** nhồi business KPI vào lớp conform
→ semantic sprawl
- **Gold-latency mismatch:** ép real-time qua batch aggregates
- **Cross-layer entanglement:** đổi 1 schema → cascade vỡ nhiều lớp

Lớp thứ 4 hành động: ML features real-time, personalization, APIs. Lý do: multi-hop Bronze→Silver→Gold thêm độ trễ, lỡ cửa sổ quyết định real-time.

Databricks định nghĩa Gold là “aggregates & features sẵn sàng cho analytics và machine learning” → Gold chính là nguồn cho Feature Store (xem phần AI-Specific Pipelines).



FTI architecture: Feature / Training / Inference — 3 pipeline độc lập, nối bằng feature store.

Đẩy data đã transform từ warehouse *ngược* về SaaS vận hành (CRM, ad tools): scores, predictions, segments. “Data activation”.

- Tools: Hightouch (warehouse-native, 300+ destinations) & Census (Audience Hub); cả hai ~\$350/mo
- Thị trường ~\$485M (2024), tăng ~35%/năm
- Động lực: phần lớn data warehouse nằm im, không được “kích hoạt” (số liệu vendor)

- **append**: chèn thêm (event logs)
- **replace**: ghi đè toàn bộ (snapshot nhỏ)
- **merge**: upsert theo primary key → dedup + cập nhật

Đọc *transaction log* của DB (không query bảng) → stream mọi insert/update/delete. Bắt được cả delete, độ trễ thấp, không tải DB nguồn.

Cursor-based: theo dõi max(updated_at); chỉ kéo bản ghi mới. Lưu cursor state giữa các lần chạy. Rẻ hơn full-extract nhiều lần.

Debezium 3.4 (12/2025) — Công cụ CDC log-based chuẩn mực: biến PostgreSQL/MySQL/Mongo thành luồng sự kiện Kafka. Tested với PostgreSQL 18.

Khi nào CDC? Cần real-time sync, cần bắt delete, hoặc không muốn batch-query làm nặng DB sản xuất.

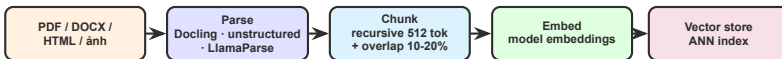
```
import dlt
# Python-native EL: schema evolution tu dong
@dlt.resource(write_disposition="merge",
              primary_key="id")
def orders(updated_after=dlt.sources
            .incremental("updated_at")):
    yield from api.get_orders(
        since=updated_after.last_value)

pipe = dlt.pipeline(destination="duckdb")
pipe.run(orders) # cursor state lưu tu dong
```

dlt 1.0 (9/2024), nay 1.28; schema evolution: evolve / freeze / discard.

Tool	Chọn khi
dlt	Python-native, code-first
Airbyte	OSS self-host, kiểm soát
Fivetran	Fully-managed, CDC tự động
Singer/Meltano	Tap/target protocol

Lưu ý: Hợp nhất thị trường: Fivetran + dbt Labs hoàn tất sáp nhập 1/6/2026 ($\approx$ \$600M ARR). EL và T đang hội tụ. Coi chừng mô hình tính phí per-connector MAR (Monthly Active Rows).



Recursive ~512-token vẫn là default mạnh nhất (~69% acc, benchmark 2025-26) — semantic chunking *không* tự động tốt hơn. **Late chunking** (embed cả doc rồi mới cắt) làm fixed $\approx$ semantic.

Hierarchical/parent-child, Contextual Retrieval, late chunking, hybrid retrieval — xây index cho *complex* RAG → §13.

Incremental indexing: chỉ re-embed doc đổi (hash/version) → 10-15% thay vì 100%. Vector/feature store sâu hơn → Ngày 19.

ai_training_pipeline DAG



Gotcha kinh điển: catchup=True (mặc định!) → deploy DAG mới = chạy **TẤT CẢ** schedule đã lỡ ngay lập tức. 30 backfill run → GPU quá tải. **Luôn set catchup=False** cho training pipeline.


```
from airflow.decorators import dag, task

@dag(schedule="0 2 * * *", catchup=False,
      max_active_runs=1)      # 1 run / lúc
def ai_training_pipeline():
    @task
    def extract():
        return list_s3_files("raw/")

    @task                      # fan-out runtime
    def process(path):
        return clean_one(path)

    @task
    def load(rows): write_gold(rows)

    files = extract()
    load(process.expand(path=files)) # .expand!

ai_training_pipeline()
```

- @task thay PythonOperator
- .expand(): tạo task instance lúc runtime (xử lý N file)
- **Deferrable operators**: nhả worker slot khi chờ; triggerer dùng asyncio giữ hàng trăm task

Lưu ý: Deferral + fan-out cực lớn ($\approx 17k$ task) có thể quá tải metadata DB. `max_active_runs=1` cho training.

- **DAG Versioning**: run hoàn tất theo đúng version lúc bắt đầu, kể cả deploy giữa chừng
- **Data Assets** (AIP-74) + @asset decorator: Datasets thành công dân hạng nhất
- **Event-driven scheduling**: DAG kích hoạt khi asset bên ngoài cập nhật
- **Client-server** (AIP-72): Task Execution API + Task SDK, task isolation

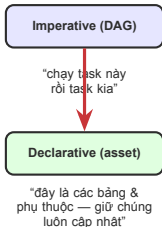
- **Edge Executor** (AIP-69): chạy task trên remote/edge compute
- **3.1 (9/2025)**: Human-in-the-Loop tasks (cổng phê duyệt cho AI/ML), Deadline Alerts (SLA)
- Stable hiện tại: **3.2.x** (data-aware workflows at scale)

Vì sao quan trọng — Airflow chuyển từ thuần “task-based” sang hỗ trợ “asset-aware” — hội tụ một phần với mô hình của Dagster (slide kế).

Công cụ	Mô hình	Chọn khi
Apache Airflow 3.x	Task-based (+ asset-aware)	Chuẩn ngành, ecosystem lớn, batch ETL
Dagster 1.12	Asset-based (SDA)	Coi data asset là trung tâm, lineage, data-quality-aware
Prefect 3.0	Task + transactions	Cần idempotency/rollback hooks, Pythonic, nhẹ
Temporal	Durable execution	Pipeline/agent chạy lâu, cần resume-on-crash
Kestra 1.x	Declarative YAML	Team đa ngôn ngữ, không muốn DAG thuần Python

Task-based (Airflow) — Bạn mô tả *thứ tự các bước* (how). “Chạy A rồi B rồi C.”

Asset-based (Dagster SDA) — Bạn khai báo *các asset & phụ thuộc* (what). Dagster tự suy ra đồ thị thực thi. Declarative Automation re-materialize theo freshness/status.



Databricks mở mã DLT engine → donate cho Apache Spark (DAIS 6/2025). **GA trong Spark 4.1.0 (16/12/2025)**. Khai báo dataset bằng decorator, Spark tự dựng đồ thị & thứ tự.

DLT → **Lakeflow Declarative Pipelines** (thương mại, GA 6/2025, code cũ chạy nguyên) và **Spark Declarative Pipelines** (OSS) — chung dòng máu.

```
from pyspark import pipelines as dp

@dp.materialized_view # batch, tính trước
def orders_clean():
    return (spark.read.table("bronze.orders")
            .dropDuplicates(["order_id"]))

@dp.table # streaming, exactly-once
def orders_stream():
    return (spark.readStream
            .table("bronze.orders_raw"))
# spark-pipelines init / dry-run / run
```

- **Materialized View:** precompute batch, cắt cost/latency truy vấn
- **Streaming Table:** xử lý mỗi record *exactly-once* trên nguồn append-only

Lưu ý: Caveat: declarative dataflow hấp thụ *orchestration* nội bộ pipeline, nhưng KHÔNG phải scheduler. Vẫn cần Airflow/Dagster cho when/trigger/history. “Airflow chưa chết.”

SDP tự phân tích phụ thuộc & orchestrate. dry-run bắt lỗi cyclic/analysis trước khi chạy.

```
import dagster as dg

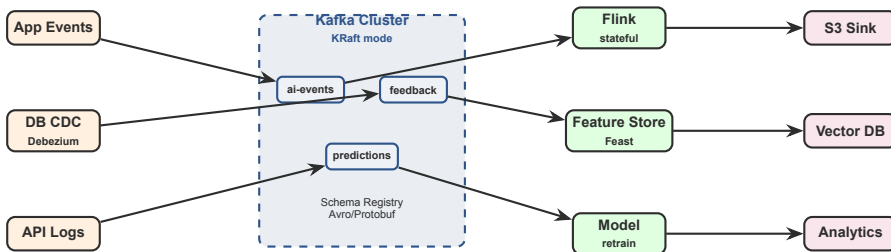
@dg.asset(group_name="silver")    # = 1 bang Silver
def orders_clean(orders_raw):    # dep = ten tham so
    return (orders_raw.dropna(subset=["order_id"])
            .drop_duplicates("order_id"))

@dg.asset(automation_condition=  # tu chay khi
        dg.AutomationCondition.eager()) # upstream doi
def orders_gold(orders_clean):
    return orders_clean.groupby("user_id").size()

@dg.asset_check(asset=orders_clean) # quality gate
def no_dupes(orders_clean):
    dup = orders_clean["order_id"].duplicated().any()
    return dg.AssetCheckResult(passed=not bool(dup))
```

Airflow: bạn viết @task & nói thứ tự (*how*).
Dagster: bạn khai báo *asset* + dependency là tên tham số — Dagster tự suy ra đồ thị (*what*).

- AutomationCondition = Declarative Automation (stable 1.12) thay cron
- asset_check = quality gate gắn vào asset — chính là quarantine/contract logic của §9/§10



Kafka 4.0 (18/3/2025): bản major đầu tiên chạy *hoàn toàn không ZooKeeper* — KRaft là mode duy nhất. Cluster cũ phải migrate qua 3.9 (bridge) trước khi lên 4.0.

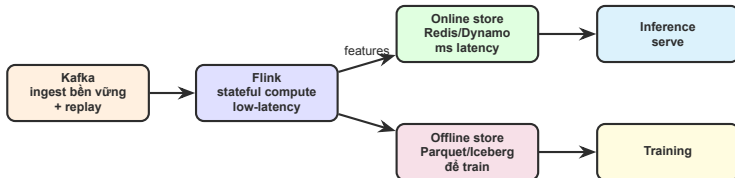
- Partition theo `user_id` → ordering mỗi user
- Partitions = consumer parallelism
- Replication factor ≥ 3 cho durability

- **WarpStream** (Confluent mua 9/2024): ghi thẳng S3, bỏ phí inter-AZ
- **KIP-1150** Diskless Topics: được *chấp nhận* vào Apache Kafka (3/2026)
- **Redpanda**: C++ thread-per-core, không JVM/ZooKeeper

Avro/Protobuf/JSON; compatibility **BACKWARD** (mặc định), FORWARD, FULL. Bảo vệ model khỏi schema drift âm thầm.

Số liệu giảm 48-90% chi phí / 10x latency là *vendor-sourced* — benchmark độc lập khiếm tốn hơn. Bối cảnh: IBM mua Confluent (~\$11B, công bố 12/2025).

Exactly-once (EOS) — Idempotent producer (PID + sequence number) + transactions. KIP-890 chống “hanging transactions”.



Vì sao streaming thắng batch cho AI features: (1) freshness từ phút/giờ xuống ms/giây; (2) **giảm training-serving skew** (feature tính khác nhau lúc train vs serve) — tính *cùng* feature liên tục, ghi vào cả online & offline store. Batch vẫn rẻ hơn cho feature ít đổi. (Feature store sâu hơn → Ngày 19.)

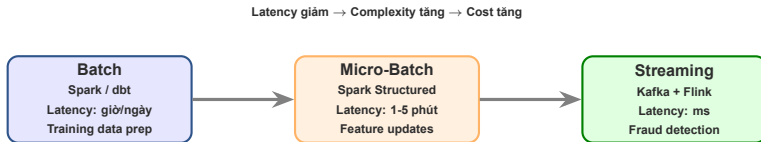
```
-- Source: Kafka topic, event-time + watermark
CREATE TABLE clicks (
  user_id STRING, amount DOUBLE, ts TIMESTAMP(3),
  WATERMARK FOR ts AS ts - INTERVAL '5' SECOND
) WITH ('connector'='kafka', 'topic'='ai-events',
      'format'='avro-confluent'); -- Schema Registry

-- Sink: online feature store (upsert by key)
CREATE TABLE feat_5m (
  user_id STRING, spend_5m DOUBLE,
  PRIMARY KEY (user_id) NOT ENFORCED
) WITH ('connector'='upsert-kafka', ...);

INSERT INTO feat_5m -- tumbling 5-min window
SELECT user_id, SUM(amount)
FROM TABLE(TUMBLE(TABLE clicks, DESCRIPTOR(ts),
                  INTERVAL '5' MINUTES))
GROUP BY user_id, window_start, window_end;
```

Đây là cái box “Flink” trong reference architecture: feature spend_5m tính liên tục, upsert vào online store → inference đọc ms-latency.

Watermark — `ts - INTERVAL '5' SECOND`: chấp nhận trễ tối đa 5s. Event trễ hơn → `late-handling.mode=filter` đẩy vào System Table \$late để reprocess.



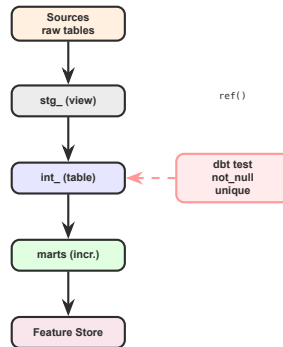
Lambda: batch + speed layer song song,
serving layer merge cả hai (mạnh nhưng phức tạp)

Kappa: streaming cho mọi thứ,
replay topic = "batch" (đơn giản hơn, trending)

Hội tụ 2025: streaming tables + materialized views (declarative) làm mờ ranh giới — một định nghĩa chạy được cả batch lẫn streaming. Kafka + Flink xử lý cả historical replay lẫn real-time (Kappa).

1. **staging** (stg_): 1:1 source, rename/cast, không join
2. **intermediate** (int_): business logic, joins
3. **marts** (fct_/dim_): consumption-ready, đúng grain

view (dev) → table (prod) → incremental (>100M rows) → materialized_view. Chiến lược **micro-batch** (GA dbt 1.9) cho time-series lớn: không cần `is_incremental()`.

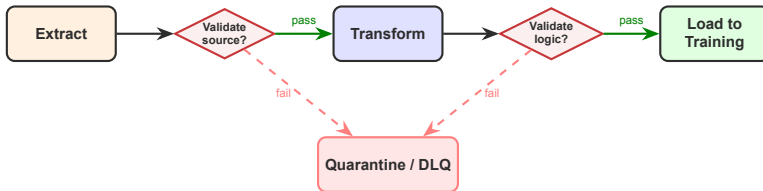


Gold marts → feature store (Feast) → model training.

```
# unit test (dbt 1.8+): test LOGIC
# voi fixtures tinh, khong can warehouse
unit_tests:
  - name: test_dedup_keeps_latest
    model: stg_orders
    given:
      - input: ref('raw_orders')
        rows:
          - {id: 1, ts: "2026-01-01"}
          - {id: 1, ts: "2026-01-02"}
    expect:
      rows:
        - {id: 1, ts: "2026-01-02"}
```

- Engine viết lại bằng **Rust** (beta 5/2025): parse nhanh tới 30x, compile 2x
- **dbt Core v2.0 (6/2026)**: nền Fusion, runtime nay *Apache 2.0*
- **Mesh**: `ref('project', 'model')` cross-project; Semantic Layer (MetricFlow)

SQLMesh (đổi thủ) — Virtual Data Environments (env qua view, zero-copy); plan/apply kiểu Terraform; column-level lineage. Fivetran donate cho Linux Foundation (3/2026).



Quarantine / DLQ (dead-letter queue) triage — 3 nhóm: **Retriable** (transient → replay sau) · **Fixable** (schema/thiếu data → kỹ sư sửa rồi replay) · **Poison** (không bao giờ qua → archive + alert). **DLQ spike = tín hiệu schema drift / quality regression.**

Tool	Granularity	Interface	Chặn pipeline bằng
Pydantic v2	Mỗi record / JSON	Typed code (Rust core, 5-50x)	Raise lúc parse từng bản ghi
Pandera 0.3x	Mỗi dataframe	Typed schema (pandas/Polars/Spark)	Schema-on-read, fail batch
GX Core 1.0	Mỗi dataset	Expectation suite + Checkpoint	Action (Slack alert / halt)
Soda Core v4	Mỗi dataset	YAML SodaCL / data contract	Contract verify trong CI

Schema-on-read enforcement — Validate cấu trúc/type lúc *consume* (Polars LazyFrame / Spark read), không chỉ lúc write. Pandera check schema-level mà không cần `collect()`.

GX Core 1.0 (8/2024): API viết lại, 47 expectations typed. Pandera multi-engine + Narwhals backend. Soda v4: data contracts là cách mặc định. *Observability runtime* (Monte Carlo, anomaly) → Ngày 27.

```
import pandera.pandas as pa

schema = pa.DataFrameSchema({
    "user_id": pa.Column(str, nullable=False),
    "label": pa.Column(str, pa.Check.isin(
        ["pos", "neg"])),
    "confidence": pa.Column(float,
        pa.Check.in_range(0.0, 1.0)),
})

try:                                # gate: fail early
    clean = schema.validate(df, lazy=True)
except pa.errors.SchemaErrors as e:
    bad = e.failure_cases           # -> quarantine
    write_quarantine(bad)          # DLQ, alert on spike
```

- Validate ngay sau extract → bắt lỗi nguồn
- Validate sau transform → bắt bug logic
- lazy=True: gom *mọi* lỗi, không dừng ở lỗi đầu

Lưu ý: Một bad record KHÔNG được làm sập cả pipeline. Tách ra quarantine, để good records chạy tiếp. Idempotent sink → replay an toàn.

Lưu ý: Gate dựa-trên-rule (GX/Pandera/Soda) mù với lỗi ngữ nghĩa — đúng kiểu sự cố hook: accuracy 94% mà conversion giảm 12%, monitor thống kê không thấy.

- Near-dup ngữ nghĩa: “NYC” vs “New York City”
- Hợp-lệ-schema nhưng vô lý: CEO sinh năm 2025
- Bất thường ngữ cảnh trong free-text / JSON / log

1. **Tier 1** SLM/rules ($\approx 80\%$ checks: schema, null, type, exact-dedup, format) — rẻ 10-30x
2. Escalate *chỉ* các dòng mơ hồ
3. **Tier 2** LLM judge: semantic dedup, cross-field reasoning, free-text anomaly

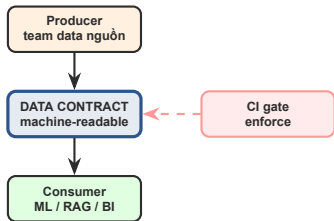
Nguyên tắc — Đừng LLM-check mọi thứ (đắt + chậm). Dùng rule làm tầng lọc, LLM cho phần rule không với tới.

Lưu ý: Với agent, data pipeline **chính là attack surface**: mọi thứ agent đọc sau này (chunk vector DB, tool output, web scrape) là vector injection. Quality gate phải mở rộng thành *security gate*.

- **PoisonedRAG**: chèn vài passage độc vào vector DB → hỏng câu trả lời RAG
- **Memory poisoning**: text độc agent scrape → ghi vào memory bền vững (“poison once, exploit forever”)
- **Tool-output poisoning**: response của tool là nội dung không tin cậy quay lại context

- **Provenance**: gắn nguồn tin-cậy cho mỗi chunk; quarantine nguồn lạ
- Tách *trusted* vs *untrusted* ngay khi ingest
- Contract + signature cho data vào agent memory

Khác Day 11 (Guardrails): control point ở đây là *data pipeline* (provenance lúc ingest), không phải prompt. “Lethal trifecta” (Simon Willison): untrusted content + private data + exfiltration.



Contract là *interface*; CI thực thi nó.

1. **Schema**: types, constraints, nesting
2. **Semantics**: ý nghĩa từng field
3. **Quality**: checks (not-null, ranges)
4. **SLA**: freshness, availability, retention

Lưu ý: Vì sao quan trọng cho AI: ngăn schema drift âm thầm từ up-stream làm hỏng training data / vỡ RAG & feature pipeline. Chất lượng được ép *tại nguồn*, không sửa downstream.

Open Data Contract Standard, Apache 2.0, ra 12/2025.
Nguồn gốc: template của PayPal (2023). Headline: **Relationships** (FK), **strict JSON Schema**, **executable SLAs**. (Data Contract Specification cũ đã deprecated → dùng ODCS.)

Bắt breaking change *trước khi deploy*, không phải sau khi model hỏng.

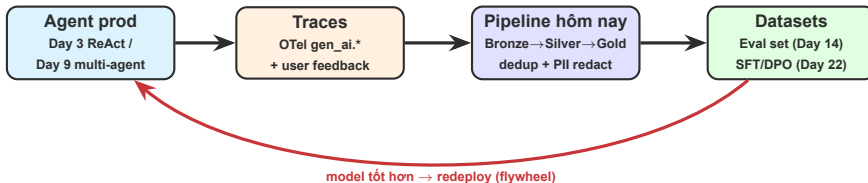
- **Gable.ai**: quét *source code* ứng dụng trong CI, chặn schema change phá vỡ contract (Series A \$20M, 3/2025)
- **Schemata**: OSS schema scoring (producer-side quality)
- Schema Registry (Kafka) = data contract cho streaming

datacontract CLI v1.0 (6/2026) — Một `datacontract.yaml` → export ra dbt models / SodaCL / DQX checks, test trên ~12 backend (Ibis engine). “Single source of truth”.

```
apiVersion: v3.0.0           # ODCS (Bitol / LF)
kind: DataContract
id: orders-gold
schema:
  - name: orders_gold
    properties:
      - name: order_id
        logicalType: string
        required: true
        unique: true          # -> exec quality check
      - name: amount
        logicalType: number
        quality:
          - rule: range
            mustBeBetween: [0, 100000]
    slaProperties:
      - property: freshness
        value: 4
        unit: h               # executable SLA
```

- datacontract test orders-gold.yaml
- datacontract changelog old new → phát hiện breaking change, block PR
- export ra dbt / SodaCL / Great Expectations

Lưu ý: Contract không còn là slogan: file này *chạy được*. Schema + quality + SLA trong một interface máy kiểm tra được — copy vào repo project của nhóm.



Trong AICB bạn đã build agent (Day 3, Day 9). Agent đó *phát ra trace*. Pipeline hôm nay biến trace thành dataset: **1 agent turn = 1 Bronze record** (prompt, tool calls, response, feedback). Day 27 là *nguồn* trace; Day 14 & Day 22 là *nơi tiêu thụ* dataset.

Lưu ý: Vì sao duplicate hại model? Model *memorize* bản ghi lặp → verbatim regurgitation, kém generalize. Memorization tăng theo số lần lặp.

Aquila2 & InternLM-2 bị phát hiện train trùng data GSM8K → điểm benchmark thối phồng (arXiv 2404.18824). Dedup + decontamination là bước pipeline, không phải tùy chọn.

- **MinHash + LSH:** near-dedup chuẩn cho corpus tỉ token
- **SemDeDup:** dedup ngữ nghĩa qua embedding → bỏ ~50% data, gần như không mất accuracy, train nhanh gấp đôi
- **FED/SEDD (2025):** GPU tăng tốc “tuần xuống giò”

Sắc thái (FineWeb) — Dedup là *empirical*: FineWeb thấy per-snapshot MinHash tốt nhất, global dedup lại hại. “Nhiều hơn” không luôn tốt hơn — đo, đừng đoán.

```
from datasketch import MinHash, MinHashLSH

def sig(text, k=5, perm=128):          # k-shingle
    m = MinHash(num_perm=perm)
    toks = text.lower().split()
    for i in range(len(toks) - k + 1):
        m.update(" ".join(toks[i:i+k]).encode())
    return m

lsh, keep = MinHashLSH(threshold=0.8, num_perm=128), []
for doc_id, text in corpus:           # 1 pass, streaming
    s = sig(text)
    if not lsh.query(s):                # không trùng ai
        lsh.insert(doc_id, s); keep.append(doc_id)
```

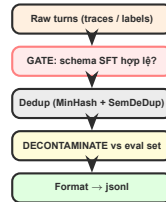
- **MinHash+LSH**: near-dup từ vựng, scale tỉ token
- **SemDeDup**: embed → cluster → bỏ cặp cosine cao → ngữ nghĩa; bỏ ~50% data gần như không mất accuracy

Chọn ngưỡng — threshold=0.8 (Jaccard): cao hơn → ít gộp; thấp hơn → gộp mạnh tay. Tune theo corpus.


```
# SFT (instruction tuning) - 1 dòng / ví dụ
{"messages": [{"role": "user", "content": "..."},
               {"role": "assistant", "content": "...turn tot..."}]}

# DPO / ORPO preference pair
{"prompt": "...",
 "chosen": "...thumbs-up tu trace...",
 "rejected": "...thumbs-down..."}
```

Nguồn "chosen/rejected" = feedback trong agent trace (flywheel).



Decontaminate: loại ví dụ trùng eval set (Day 14) khỏi train set →
chống điểm thổi phồng.

Training-serving skew là nguyên nhân #1 model de-grade trong production. Feature store tồn tại chủ yếu để diệt nó: định nghĩa feature một lần, serve giống hệt cho train (offline) & inference (online).

lakeFS mua lại dự án OSS **DVC** (11/2025) — hợp nhất 2 công cụ version-control dữ liệu. Git-cho-data: reproduce training set chính xác.

Point-in-time correctness — As-of join theo event timestamp: mỗi training row chỉ thấy feature *có sẵn tại thời điểm đó* → chống label leakage. Đặc thù AI, không có trong ETL analytics cổ điển.

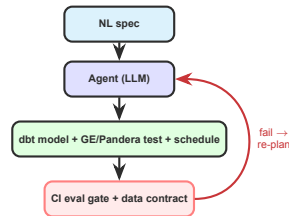
Model collapse có thật, nhưng failure mode là *thay thế* data thật, không phải synthetic per se. Giữ neo data thật cố định + *tích lũy* synthetic → tránh collapse.

Mô hình FTI (Feature/Training/Inference): Feast (PyTorch Ecosystem 2025, + vector store cho RAG), Chronon (Airbnb, OSS; Stripe/OpenAI/Netflix/Roku/Uber).

Kiến trúc feature/vector store sâu hơn → Ngày 19.

Retraining 2025 — Từ lịch cố định → **drift-triggered** + PEFT/LoRA rolling update, gate bằng eval tự động trước khi deploy.

1. **Copilot:** dbt Copilot / SQLMesh AI sinh model + test từ NL, người review diff
2. **Agent dựng pipeline:** từ 1 câu (“pull 7 ngày Shopify, flatten JSON, ghi BigQuery”) → tạo dbt model + test + schedule Airflow
3. **Self-healing DAG:** agent đọc log task fail + lineage → đề xuất patch → mở PR



Bạn sẽ *giám sát* agent, không chỉ viết DAG. Cái làm nó an toàn: **eval gate + data contract** của chính bài hôm nay.

```
# 1 agent turn (OTel gen_ai span) -> 1 Bronze row
def span_to_row(span):
    a = span.attributes
    return {
        "trace_id": span.context.trace_id,
        "ts": span.start_time,
        "model": a["gen_ai.request.model"],
        "prompt": a["gen_ai.prompt"],
        "response": a["gen_ai.completion"],
        "tool_calls": a.get("gen_ai.tool.calls"),
        "tok_in": a["gen_ai.usage.input_tokens"],
        "tok_out": a["gen_ai.usage.output_tokens"],
        "feedback": a.get("user.feedback"), # up/down
    }

# OTLP / Kafka -> append-only Bronze (partition by day)
write_bronze([span_to_row(s) for s in spans])
```

Trace của agent Day 3/9 thành Bronze record. Schema theo OTel `gen_ai.*` sem-conv (GD1 Day 13); Day 27 là observability runtime.

- Silver: PII redact + dedup theo `trace_id`
- Partition theo ngày → incremental
- Gold: turn sạch cho eval / fine-tune

```
@dag(schedule="@hourly", catchup=False)
def rag_ingest():
    @task
    def discover():
        # incremental, KHÔNG full
        return [d for d in list_docs()
                if hash_changed(d)] # chỉ doc doi
    @task
    # .expand: fan-out per doc
    def parse_embed(doc):
        chunks = recursive_split(parse(doc), 512)
        ok, bad = validate(chunks) # quarantine
        write_quarantine(bad)
        return [{"id": cid(doc, c), "vec": embed(c),
                "source": doc.uri} for c in ok]
    @task
    def upsert(rows):
        # idempotent: upsert by id
        vectordb.upsert(rows) # re-run an toàn
    upsert(parse_embed.expand(doc=discover()))
```

Frame §3 vẽ doc→chunk→embed phẳng. Đây là phiên bản *orchestrated*: dùng primitive của chính ngày hôm nay.

- **Incremental**: re-embed theo content hash → 10-15%, không 100%
- **Idempotent**: upsert by id, replay an toàn
- source = provenance cho security gate

Vector store sâu hơn → Ngày 19.

```
# Gold mart (dbt) -> Feast feature view
user_fv = FeatureView(
    name="user_features", entities=[user],
    schema=[Field("orders_7d", Int64),
            Field("avg_order_value", Float32)],
    source=gold_user_features, # bang Gold của Lab 17
    ttl=timedelta(days=2))

# OFFLINE (train): point-in-time correct
store.get_historical_features(
    entity_df, features).to_df()

# ONLINE (agent quyết định, ms latency)
store.get_online_features(
    features, [{"user_id": uid}])
```

Một FeatureView phục vụ cả train (offline) lẫn agent inference (online) → diệt training-serving skew.

- Gold (Lab 17) → source
- feature_ts → point-in-time
- materialize incremental

Feature store dựng ở → Ngày 19.

```
# Prod traces -> eval set (Day14) + DPO (Day22)
def curate(traces):
    golden, prefs = [], []
    sample = stratified_sample(           # không bias
        traces, by="intent", n=500)
    for t in sample:
        score = llm_judge(t.prompt, t.response)
        golden.append({**t, "label": score}) # eval
        if t.feedback == "down" and t.regenerated:
            prefs.append({                 # DPO pair
                "prompt": t.prompt,
                "chosen": t.regenerated,
                "rejected": t.response})
    prefs = decontaminate(prefs, eval_set=golden)
    return golden, prefs
```

- **Golden eval set** (Day 14): label bằng LLM-judge, freeze
- **DPO pairs** (Day 22): chosen = turn người dùng sửa / thumbs-up

Lưu ý: Decontaminate: gỡ ví dụ trùng eval set khỏi train set — nếu không, điểm Day 14 vô nghĩa.

```
import webdataset as wds

# Lakehouse Gold -> sharded tar cho multi-GPU
# moi GPU stream 1 slice riêng (khong shuffle ca set)
ds = (wds.WebDataset(
    "s3://gold/shards/{000..255}.tar",
    shardshuffle=True, nodesplitter=wds.split_by_node)
    .shuffle(1000).decode().to_tuple("json"))

loader = DataLoader(ds, batch_size=32,
                    num_workers=8, pin_memory=True)
# streaming: khong materialize toan bo dataset vao RAM
for batch in loader:
    train_step(batch)
```

Output của pipeline phải *nạp được* vào GPU nhanh — nếu không, GPU đổi data, lãng phí \$\$.

- Shard **256-512MB** (đúng small-files target §14!)
- **Streaming**: không load cả set vào RAM
- `num_workers` + prefetch overlap I/O với GPU compute
- Shard theo node → no cross-GPU shuffle bottleneck

Model serving / inference → Ngày 20.

Lưu ý: PDF thực tế *bản*: bảng, hình, multi-column, scan. Parse phẳng `read_text()` → lộn cột, mất bảng → phá retrieval ngay từ ingest.

Một vision-language model làm cả layout + reading-order + OCR + bảng → markup có cấu trúc (DocTags). Thay pipeline OCR nhiều tầng.

- **Granite-Docling-258M** (IBM, 9/2025, Apache 2.0): nhỏ, chạy local
- Surya 2 / Marker, dots.ocr, olmOCR 2

Parser	Đặc điểm
Docling / Granite-Docling LlamaParse unstructured.io ColPali	bảng phức tạp 97.9%; OSS, local nhanh (~6s) nhưng multi-column dễ lộn rộng định dạng, automation-first <i>bỏ qua parsing</i> (frame kế)

Lưu ý: Cảnh báo benchmark: phần lớn số liệu là vendor *tự chấm*. Con số 97.9% (Docling) từ Procycons — bên thứ ba trung lập. Tự đo trên data của bạn.

Embed child nhỏ (100-500 tok) để *tìm chính xác*, trả parent lớn (500-2000 tok) cho LLM để *đủ ngữ cảnh*. Parent $\approx 3-5 \times$ child. (LangChain ParentDocumentRetriever / LlamaIndex.)

Embed *cả doc* (long-context embedder, 8k tok) rồi mới cắt + pool $\rightarrow$ mỗi chunk “*thấy*” toàn doc, *không* tốn LLM/chunk.

Thêm blurb ngữ cảnh do LLM sinh vào mỗi chunk *trước* index $\rightarrow$ retrieval-failure -49% (-67% kèm rerank). Khả thi nhờ prompt caching: $\sim \$1.02$ / triệu token doc.

Ingest dựng **2 index** trên cùng chunk: dense (vector) + sparse (BM25/SPLADE) $\rightarrow$ merge bằng RRF $\rightarrow$ cross-encoder rerank. BM25 thắng từ hiếm (SKU, mã lỗi); dense thắng paraphrase.

Semantic/proposition chunking: *chưa* có consensus 2026 là tốt hơn recursive — đo, đừng mặc định.

Đừng parse text. Encode *ảnh trang* thành ~1024 patch-embedding (SigLIP + PaliGemma), match kiểu ColBERT MaxSim. **Bỏ qua OCR/parsing hoàn toàn** — giữ nguyên bảng, chart, layout.

Khi nào dùng — Doc giàu hình ảnh: scan, chart, form, slide — nơi parser text làm hỏng cấu trúc. ViDoRe v2 (3/2025) là benchmark hiện tại (đừng trộn điểm v1/v2).

Lưu ý: Cái giá ở ingest = **storage blowup**: ~1024 vector/trang (1 bậc lớn hơn BM25, 2 bậc hơn single-vector).

- **Binarization**: 128 float → 16 byte (~32× nhỏ hơn)
- **Token/patch pooling**; HPC-ColPali (centroid)
- Vespa / Qdrant scale tới tỉ trang

Multi-vector đổi bài toán index, không chỉ retrieval → Ngày 19.

```
from langchain_experimental.graph_transformers \
    import LLMGraphTransformer

tf = LLMGraphTransformer(
    llm=llm,
    allowed_nodes=["Person", "Company", "Product"],
    allowed_relationships=["WORKS_AT", "BUILDS"],
) # schema rang buoc -> graph sach hon
docs = chunk(parse(raw_docs)) # complex parse
graph_docs = tf.convert_to_graph_documents(docs)
neo4j.add_graph_documents( # upsert nodes/rels
    graph_docs, include_source=True) # giu provenance
```

1. Parse → chunk
2. LLM extract node + relation (theo schema)
3. **Entity resolution**: gộp “IBM” = “I.B.M.” (phần khó nhất)
4. Incremental merge (đừng rebuild)

Neo4j LLM KG Builder (Leiden communities, 2/2025) · LlamaIndex PropertyGraphIndex (graph + vector trong 1 index).

LLM extract entity → **Leiden** communities → LLM tóm tắt community. Query: **local** (quanh entity), **global** (map-reduce mọi community), **DRIFT** (cân bằng).

- **Multi-hop reasoning** (nối nhiều mẫu)
- **Summarization / global sense-making** (“chủ đề chính của cả corpus?”)

Lưu ý: Khi nào graph **THUA**: fact lookup đơn giản — basic RAG 60.9% vs GraphRAG 49.3%. **Không phải upgrade miễn phí.**

Lưu ý: **Cái giá = token**: global search ~331k token/-query vs vanilla ~879.

- **LazyGraphRAG**: index 0.1%, query rẻ >700×
- **LightRAG**: incremental update (không re-index)
- **fast-graphrag**: PageRank thay map-reduce

Loại pipeline	Pattern idempotent	Cơ chế
Batch	Overwrite-partition	Ghi đè cửa sổ reprocess (không append)
Merge / CDC	Upsert theo natural key	order_id, txn_id
Event-driven	Dedup theo idempotency key	Audit log các event_id đã xử lý

Lưu ý: Bất biến: pipeline phải an toàn khi chạy lại / replay / backfill *bao nhiêu lần cũng được* mà không nhân đôi rows, không double-charge, không double-send.

Exponential backoff (Airflow 3.2: multiplier cấu hình được, không còn cứng x2), bounded max_retry_delay. **Calibrate theo rate-limit upstream** để tránh thundering-herd. Replay & backfill là first-class ops.

Lưu ý: **Small-files problem** là failure mode cost+reliability số 1 của lakehouse. Target **256-512MB/file** Parquet; file <128MB → metadata bloat + query-planning chậm. Compaction (rewrite/optimize của table format) định kỳ là trách nhiệm scheduling của pipeline; cơ chế → Ngày 18.

2 loại: **completion-rate** (“99% job hoàn tất”) & **freshness** (“data cập nhật trong 4h”). Tier theo độ quan trọng (vd: 99.9% payments / 99.0% analytics — con số minh họa).

Paxos giảm ~50% chi phí data-platform khi chuyển sang incremental models (báo cáo của Paxos). Cluster nhỏ hơn, chạy thường xuyên hơn. **Compute (không phải storage)** chi phối hoá đơn warehouse.

Late data (streaming) — Watermark theo event-time; `forBoundedOutOfOrderness()` đặt max-lateness. Confluent Flink: `late-handling.mode=filter` đẩy late events vào System Table `$late` để reprocess.

FinOps / GPU cost sâu hơn → Ngày 25. SLO & incident response → Ngày 27.

```
# 1. INGEST (disposition: merge / replace / append)
raw = ingest(source, primary_key="id",
             disposition="merge") # dlt / CDC / Kafka
write("bronze.events", raw)      # giu RAW bat bien

# 2. DEDUP @ Silver (idempotent: upsert by key)
silver = dedup(read("bronze.events"), key="id")

# 3. GATE (fail early -> quarantine, KHONG sap)
ok, bad = schema.validate(silver, lazy=True)
write_quarantine(bad); alert_on_spike(bad)

# 4. TRANSFORM (dbt: stg_ -> int_ -> marts; test)
gold = dbt_build(select="tag:ml_features")

# 5. SCHEDULE (catchup=False, max_active_runs=1)
```

- Train data → **batch** ELT + Medallion
- Real-time feature → **Kafka + Flink**
- Cần bắt delete / low-lag sync → **CDC**
- Nhiều file nhỏ → compaction
256-512MB
- Cost cao → **incremental** models

Lưu ý: Bắt buộc: an toàn để re-run/replay/backfill bao nhiêu lần cũng được. Đây chính là Lab 17.

1. **Bước 1 — Orchestrate:** DAG ingest S3/DuckDB → Bronze → Pandera gate → dbt run → Gold
2. **Bước 2 — Dedup ở Silver:** inject 30% duplicate records → dedup theo key → đếm rows giảm
3. **Bước 3 — Bad data:** inject record sai schema → gate fail → quarantine/DLQ kích hoạt
4. **Bước 4 — dbt test:** dbt test (not_null/unique/unit test) pass; dbt docs serve xem lineage
5. **Bước 5 — Streaming sim:** producer events → consumer cập nhật feature; (bonus) doc → chunk → embedding

Những ý chính cần nhớ trước khi sang bài tiếp theo

1

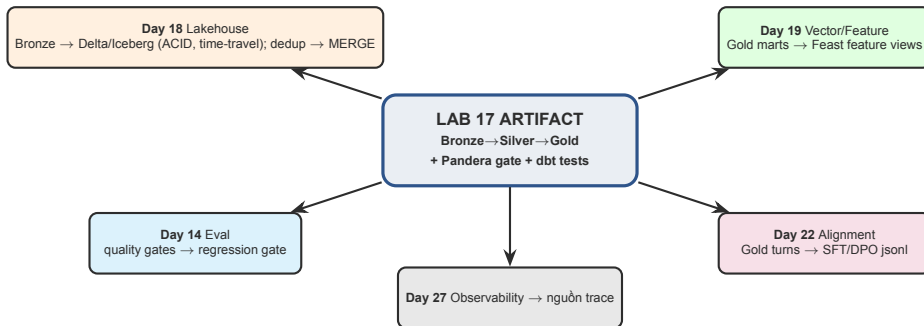
Pipeline là một *dependency của model*: Medallion (Bronze/Silver/Gold), luôn giữ raw, dedup ở Silver. Pipeline hỏng âm thầm = model hỏng âm thầm.

2

Orchestrate (Airflow 3 / asset-based Dagster) + ingest (dlt/CDC/Kafka) + transform (dbt/declarative) + **validation gates** = stack production-ready cho AI data.

3

Idempotency là tiền đề cho retry/replay/backfill an toàn. Data contracts ép chất lượng tại nguồn. Fail early, quarantine bad records, save compute.



Output của Lab 17 là *input* của các ngày sau. Course tích lũy: làm tốt pipeline hôm nay → mọi ngày sau nhẹ hơn.

Ngày 18: Data Lakehouse Architecture

“Delta Lake, Apache Iceberg, ACID transactions, time travel — nơi data pipeline đổ vào table format mở”

- Hoàn thành Lab 17: pipeline ingest→validate→transform→load (dedup + gate + dbt test)
- Đọc trước: Delta Lake & Apache Iceberg documentation
- Suy nghĩ: bài toán data storage cho project nhóm

Mục tiêu: Xây pipeline Medallion chạy được: DuckDB Bronze→Silver→Gold với dedup, Pandera quality gate + quarantine, dbt-duckdb models pass test, orchestrate bằng DAG thuần Python (lite) hoặc Airflow (Docker bonus).

Deliverable: Pipeline + dbt project + validation suite trong repo, chạy zero-key trên mock/DuckDB.

Thời gian: Lite path: chỉ cần pip install. Docker bonus: Airflow + Redpanda/Kafka.

Hỏi & Đáp

Câu hỏi về ETL/ELT, orchestration, Kafka, validation, data contracts, hay AI pipelines?



Cảm ơn!

AICB-P2T2 · Ngày 17

Data Pipeline Engineering

`lms.vinuni.edu.vn` · Slide & template trên LMS